МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №5

"Обробка помилок, файли та серіалізація"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

проф. Свелеба С. А.

Львів-2026

Мета роботи: Сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних. Опанувати механізм exceptions, конструкції try, except, else, finally, raise, створення custom exceptions, exception chaining, принципи EAFP і fail-fast, context managers, оператор with, власні context managers, модуль logging, pathlib, роботу з CSV, JSON, YAML, серіалізацію та десеріалізацію, конфігураційні файли, streaming I/O, атомарний запис файлів та валідацію зовнішніх даних на прикладі системи обліку студентів (варіант 1).

Теоретичні відомості:

У сучасному професійному Python створення надійних програм вимагає комплексного підходу до обробки помилок, керування ресурсами та серіалізації даних. Механізм винятків (exceptions) базується на конструкціях try, except, else, finally та операторі raise. Перехоплення завжди має бути максимально конкретним; використання загального except: pass заборонено, оскільки воно маскує помилки синтаксису та збої виконання. Для предметної області створюється власна ієрархія класів, успадкованих від базового ApplicationError. Механізм exception chaining (raise NewException from error) забезпечує збереження первинного системного винятку як атрибута __cause__, гарантуючи збереження повного контексту в логах. Парадигма EAFP (Easier to Ask for Forgiveness than Permission) поєднується з принципом fail-fast, який вимагає раннього виявлення некоректної конфігурації до запуску тривалих обчислень. Для управління ресурсами застосовується протокол context manager (__enter__, __exit__) та оператор with, що забезпечують закриття файлів навіть при виникненні аварійних ситуацій. Сучасні файлові операції використовують pathlib.Path та обов'язкове явне задання кодування encoding='utf-8'. Стандартні формати представлено модулями csv (reader, DictReader, DictWriter), json (dump, dumps, load, loads) та безпечним синтаксичним аналізатором yaml.safe_load(). Для діагностики використовується модуль logging із налаштованими рівнями серйозності та одночасним спрямуванням у файл і термінал без витоку конфіденційних даних. Потокова обробка (streaming I/O) за допомогою генераторів зберігає константне використання пам'яті O(1) для великих файлів. Політика валідації може бути суворою (strict) або толерантною (tolerant). Для уникнення пошкодження цільових файлів використовується атомарний запис (atomic write) через проміжний .tmp-файл і неподільну заміну os.replace().

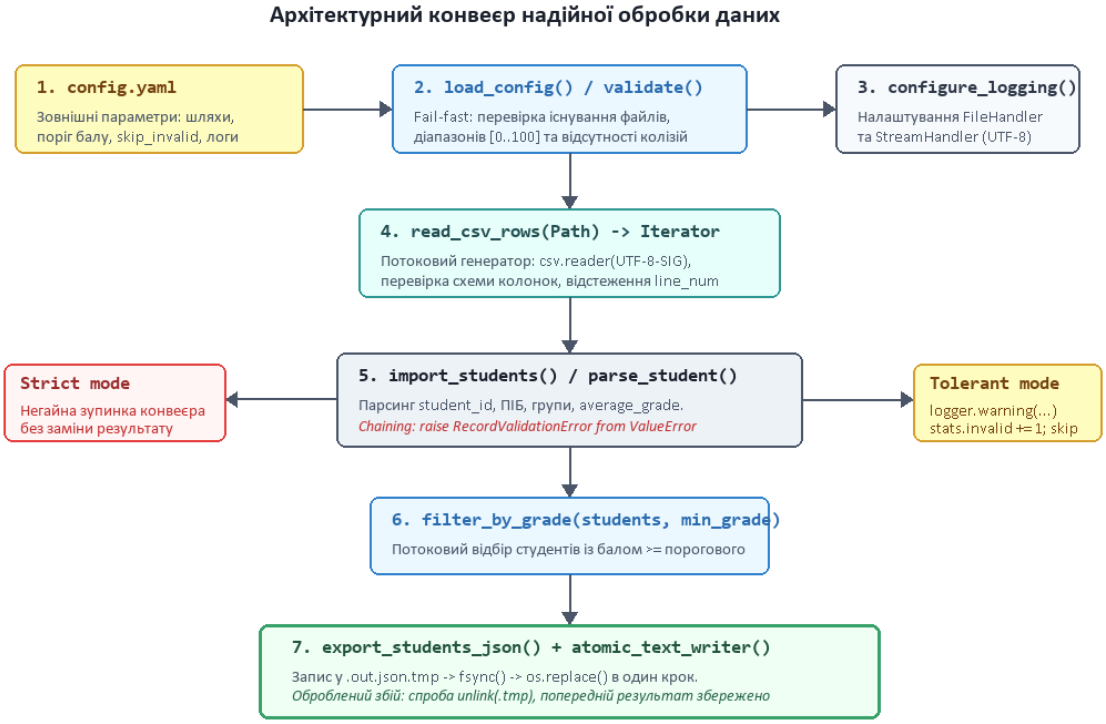
Виконання роботи

1. Постановка завдання індивідуального варіанта 1 та канонічна схема вхідних даних. Відповідно до індивідуального варіанта 1 розробив надійний модуль імпорту, валідації, конфігурації та експорту даних системи обліку студентів.

Починаючи з лабораторної роботи №3, для збереження єдності предметної області використовується строго визначена канонічна схема заголовка CSV-файлу: student_id,first_name,last_name,group,average_grade. Нижче наведено зразок вхідного навчального файлу sample_students.csv, у якому перші 3 записи є валідними, а подальші навмисно містять помилки (пусте ім'я в рядку 4, оцінка 105.0 у рядку 5, нечисловий ID 'abc' у рядку 6) для комплексної перевірки механізмів надійності.

```
student_id,first_name,last_name,group,average_grade
1,Олена,Коваль,ФеП-33,91.5
2,Іван,Франко,ФеП-33,84.0
3,Марія,Шевченко,ФеП-33,95.2
4,,Бойко,ФеП-33,76.0
5,Тарас,Новак,ФеП-33,105.0
abc,Оксана,Мельник,ФеП-33,85.0
7,Андрій,Литвин,ФеП-33,89.4
```

2. Модульна структура та архітектура конвеєра надійної обробки даних. Проєкт організовано за схемою src-layout у пакеті student_manager. Модулі розподілено за чіткими зонами відповідальності:
- models.py: датаклас Student із валідацією полів та ідентифікатора у __post_init__;
- exceptions.py: доменна ієрархія винятків на базі ApplicationError;
- config.py: завантаження та валідація YAML-конфігурації за принципом fail-fast;
- reliable_io.py: потокове читання CSV, фільтрація, серіалізація в JSON, логування та атомарний експорт;
- lab5_demo.py: консольна точка входу CLI із поверненням контрольованих кодів завершення;
- reliable_experiments.py: автономний модуль проведення обов'язкових експериментів 1-5. На рисунку нижче наведено структурну схему конвеєра обробки даних.



3. Доменна модель Student у models.py. Модель реалізовано через dataclass із валідацією параметрів у методі __post_init__. Текстові поля first_name, last_name та group очищуються від пробільних символів і перевіряються на непорожність. Середній бал average_grade обмежується допустимим діапазоном від 0.0 до 100.0. Числовий student_id перевіряється на строгий тип int (із відсіканням bool) та строгу додатність (> 0). Властивість full_name формує повне ім'я студента.

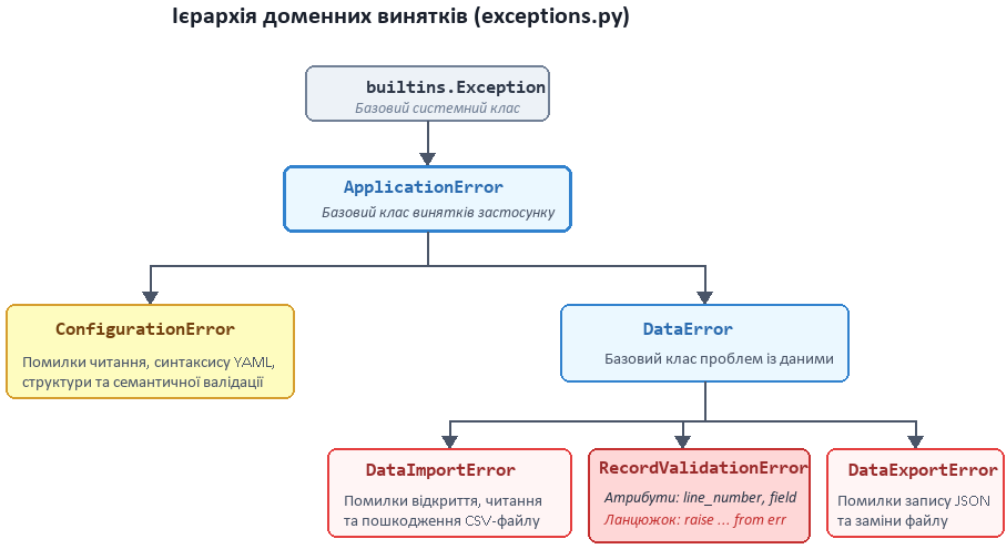
```python
from dataclasses import dataclass


@dataclass
class Student:
    first_name: str
    last_name: str
    group: str
    average_grade: float
    student_id: int | None = None

    def __post_init__(self) -> None:
        self.first_name = self.first_name.strip()
        self.last_name = self.last_name.strip()
        self.group = self.group.strip()
        if not self.first_name or not self.last_name or not self.group:
            raise ValueError("Name, surname and group cannot be empty")
        if not 0 <= self.average_grade <= 100:
            raise ValueError("Average grade must be between 0 and 100")
        if self.student_id is not None:
            # перевіряємо шоб айді був цілим додатнім числом і не bool
            if isinstance(self.student_id, bool) or not isinstance(
                self.student_id, int
            ):
                raise TypeError("Student ID must be an integer")
            if self.student_id <= 0:
                raise ValueError("Student ID must be a positive integer")

    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"
```

4. Доменна ієрархія винятків у exceptions.py. Для забезпечення надійної діагностики створено власну ієрархію класів, успадкованих від базового ApplicationError. Це дозволяє системі розрізняти помилки конфігурації (ConfigurationError) від проблем із даними (DataError). Помилки даних поділяються на збої відкриття файлів (DataImportError), валідацію конкретних рядків (RecordValidationError) та помилки експорту (DataExportError). Клас RecordValidationError містить додаткові контекстні атрибути line_number та field.




```python
"""ієрархія винятків для надійної обробки даних"""

from typing import Any


class ApplicationError(Exception):
    """базовий клас для всіх винятків застосунку"""


class ConfigurationError(ApplicationError):
    """помилка завантаження або валідації конфігурації"""


class DataError(ApplicationError):
    """базовий виняток для проблем із даними"""


class DataImportError(DataError):
    """помилка відкриття або читання вхідного файлу"""


class RecordValidationError(DataError):
    """помилка валідації конкретного рядка даних"""

    def __init__(
```

```
        self,
        message: str,
        *,
        line_number: int | None = None,
        field: str | None = None,
        **kwargs: Any,
    ) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field


class DataExportError(DataError):
    """помилка запису або заміни вихідного файлу"""
```

5. Валідація конкретних рядків і зв'язування винятків (exception chaining) у reliable_io.py. У функції parse_student кожен рядок розбирається окремо. Під час перетворення типів рядків на числа використовується конструкція raise RecordValidationError(...) from err. Завдяки цьому середовище Python зберігає вихідний системний виняток ValueError як атрибут __cause__, гарантуючи відображення повного контексту в логах.

```
def parse_student(row: dict[str, str], *, line_number: int) -> Student:
    # перевіряємо чи рядок мав правильну кількість колонок
    if "_width_error" in row:
        raise RecordValidationError(
            f"Malformed row width: {row['_width_error']}",
            line_number=line_number,
            field="row",
        )

    # перевірка student_id
    raw_id = row.get("student_id")
    if raw_id is None or not str(raw_id).strip():
        raise RecordValidationError(
            "Student ID cannot be empty",
            line_number=line_number,
            field="student_id",
        )
    try:
        student_id = int(str(raw_id).strip())
    except (ValueError, TypeError) as err:
        raise RecordValidationError(
            f"Invalid student ID '{raw_id}': must be an integer",
            line_number=line_number,
            field="student_id",
        ) from err
    if student_id <= 0:
        raise RecordValidationError(
            f"Student ID must be a positive integer, got {student_id}",
            line_number=line_number,
            field="student_id",
        )
```

```
    # перевірка first_name
    first_name = (row.get("first_name") or "").strip()
    if not first_name:
        raise RecordValidationError(
            "First name cannot be empty",
            line_number=line_number,
            field="first_name",
        )

    # перевірка last_name
    last_name = (row.get("last_name") or "").strip()
    if not last_name:
        raise RecordValidationError(
            "Last name cannot be empty",
            line_number=line_number,
            field="last_name",
        )

    # перевірка group
    group = (row.get("group") or "").strip()
    if not group:
        raise RecordValidationError(
            "Group cannot be empty",
            line_number=line_number,
            field="group",
        )

    # перевірка average_grade
    raw_grade = row.get("average_grade")
    if raw_grade is None or not str(raw_grade).strip():
        raise RecordValidationError(
            "Average grade cannot be empty",
            line_number=line_number,
            field="average_grade",
        )
```

```
try:
    grade = float(str(raw_grade).strip())
except (ValueError, TypeError) as err:
    raise RecordValidationError(
        f"Invalid grade '{raw_grade}': must be numeric",
        line_number=line_number,
        field="average_grade",
    ) from err

if not math.isfinite(grade):
    raise RecordValidationError(
        f"Average grade must be a finite number, got {raw_grade}",
        line_number=line_number,
        field="average_grade",
    )
if not 0.0 <= grade <= 100.0:
    raise RecordValidationError(
        f"Average grade must be between 0 and 100, got {grade}",
        line_number=line_number,
        field="average_grade",
    )

return Student(
    first_name=first_name,
    last_name=last_name,
    group=group,
    average_grade=grade,
    student_id=student_id,
)
```

Для експериментального підтвердження зв'язування винятків розроблено функцію run_exception_chaining_experiment у reliable_experiments.py, яка штучно передає нечисловий бал 'wrong' і фіксує ланцюжок винятків.

```
def run_exception_chaining_experiment() -> dict[str, Any]:
    row = {
        "student_id": "1",
        "first_name": "Anna",
        "last_name": "Koval",
        "group": "PI-21",
        "average_grade": "wrong",
    }
    captured_traceback = ""
    cause_repr = ""
    error_repr = ""

    try:
        parse_student(row, line_number=14)
    except RecordValidationError as error:
        error_repr = repr(error)
        cause_repr = repr(error.__cause__)
        captured_traceback = "".join(traceback.format_exception(error))

    return {
        "record": row,
        "error": error_repr,
        "cause": cause_repr,
        "has_chain": cause_repr != "None",
        "traceback": captured_traceback,
    }
```

Під час виконання експерименту 3 над записом із нечисловим балом 'wrong' зафіксував реальний стек викликів (traceback). У виведенні чітко простежується збереження первинного винятку ValueError та його перетворення на доменний RecordValidationError.

```
PS F:\university\3yr-1\professional_python> python -c "from student_manager.reliable_experiments import
run_exception_chaining_experiment; print(run_exception_chaining_experiment()['traceback'])"
Traceback (most recent call last):
  File "F:\university\3yr-1\professional_python\src\student_manager\reliable_io.py", line 109, in
parse_student
    grade = float(str(raw_grade).strip())
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: could not convert string to float: 'wrong'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "F:\university\3yr-1\professional_python\src\student_manager\reliable_experiments.py", line 196, in
run_exception_chaining_experiment
    parse_student(row, line_number=14)
  File "F:\university\3yr-1\professional_python\src\student_manager\reliable_io.py", line 111, in
parse_student
    raise RecordValidationError(
student_manager.exceptions.RecordValidationError: Invalid grade 'wrong': must be numeric
```

6. Завантаження конфігурації YAML та валідація за принципом fail-fast у config.py. Модуль config.py використовує безпечний парсер yaml.safe_load() для захисту від виконання довільного коду. Конфігурація трансформується у незмінні (frozen, slots) структури AppConfig, ProcessingConfig та LoggingConfig. Функція validate_config перевіряє існування вхідного файлу через Path.exists(), діапазон порогового бала [0..100], допустимість рівня логування та забороняє

збіг шляхів (collision protection). Функція load_config здійснює повну структурну валідацію обов'язкових секцій та їх значень.

```python
@dataclass(frozen=True, slots=True)
class ProcessingConfig:
    minimum_grade: float
    skip_invalid: bool


@dataclass(frozen=True, slots=True)
class LoggingConfig:
    level: str
    path: Path


@dataclass(frozen=True, slots=True)
class AppConfig:
    input_path: Path
    output_path: Path
    processing: ProcessingConfig
    logging: LoggingConfig
```

```python
def validate_config(config: AppConfig) -> None:
    # перевіряємо поріг оцінки
    min_grade = config.processing.minimum_grade
    if not isinstance(min_grade, (int, float)) or isinstance(min_grade, bool):
        raise ConfigurationError("processing.minimum_grade must be a number")
    if not math.isfinite(min_grade) or not 0 <= min_grade <= 100:
        raise ConfigurationError(
            "processing.minimum_grade must be a finite number between 0 and 100"
        )

    # перевіряємо булевий прапорець
    if type(config.processing.skip_invalid) is not bool:
        raise ConfigurationError(
            "processing.skip_invalid must be a boolean (true or false)"
        )

    # перевіряємо рівень логування
    lvl = config.logging.level.upper()
    if lvl not in VALID_LOG_LEVELS:
        raise ConfigurationError(
            f"logging.level '{config.logging.level}' is invalid; must be one of {sorted(VALID_LOG_LEVELS)}"
        )
```

```python
    # перевіряємо існування вхідного файлу
    if not config.input_path.exists():
        raise ConfigurationError(f"Input file does not exist: {config.input_path}")
    if not config.input_path.is_file():
        raise ConfigurationError(
            f"Input path must be a regular file: {config.input_path}"
        )

    # забороняємо збіг шляхів щоб не затерти вхідні дані чи логи
    in_res = config.input_path.resolve()
    out_res = config.output_path.resolve()
    log_res = config.logging.path.resolve()

    if in_res == out_res:
        raise ConfigurationError(f"Input path and output path collide: {in_res}")
    if in_res == log_res:
        raise ConfigurationError(f"Input path and log path collide: {in_res}")
    if out_res == log_res:
        raise ConfigurationError(f"Output path and log path collide: {out_res}")
```

```python
def load_config(path: str | Path) -> AppConfig:
    config_path = Path(path)
    try:
        with config_path.open("r", encoding="utf-8") as file:
            raw_data = yaml.safe_load(file)
    except FileNotFoundError as err:
        raise ConfigurationError(
            f"Configuration file not found: {config_path}"
        ) from err
    except (OSError, UnicodeDecodeError) as err:
        raise ConfigurationError(
            f"Cannot read configuration file {config_path}: {err}"
        ) from err
    except yaml.YAMLError as err:
        raise ConfigurationError(
            f"Invalid YAML configuration syntax in {config_path}: {err}"
        ) from err

    if not isinstance(raw_data, dict):
        raise ConfigurationError(
            f"Configuration file root must be a YAML mapping (dict), got {type(raw_data).__name__}"
        )

    # перевіряємо обов'язкові секції
    required_sections = ("input", "output", "processing", "logging")
    for section in required_sections:
        if section not in raw_data or not isinstance(raw_data[section], dict):
            raise ConfigurationError(
                f"Configuration is missing required section '{section}' or it is not a mapping"
            )

    base_dir = config_path.resolve().parent
```

```python
    # валідація секції input
    if "path" not in raw_data["input"]:
        raise ConfigurationError("Missing 'path' in 'input' section")
    input_path = _resolve_relative_path(
        base_dir, raw_data["input"]["path"], "input.path"
    )

    # валідація секції output
    if "path" not in raw_data["output"]:
        raise ConfigurationError("Missing 'path' in 'output' section")
    output_path = _resolve_relative_path(
        base_dir, raw_data["output"]["path"], "output.path"
    )

    # валідація секції processing
    proc_dict = raw_data["processing"]
    if "minimum_grade" not in proc_dict:
        raise ConfigurationError("Missing 'minimum_grade' in 'processing' section")
    if "skip_invalid" not in proc_dict:
        raise ConfigurationError("Missing 'skip_invalid' in 'processing' section")

    raw_min_grade = proc_dict["minimum_grade"]
    if type(raw_min_grade) not in (int, float) or isinstance(raw_min_grade, bool):
        raise ConfigurationError(
            "processing.minimum_grade must be a number (float or int), not boolean or string"
        )
    min_grade = float(raw_min_grade)

    raw_skip = proc_dict["skip_invalid"]
    if type(raw_skip) is not bool:
        raise ConfigurationError(
            "processing.skip_invalid must be a boolean (true or false), not string or int"
        )
```

```
    # валідація секції logging
    log_dict = raw_data["logging"]
    if "level" not in log_dict:
        raise ConfigurationError("Missing 'level' in 'logging' section")
    if "path" not in log_dict:
        raise ConfigurationError("Missing 'path' in 'logging' section")

    raw_level = log_dict["level"]
    if not isinstance(raw_level, str) or isinstance(raw_level, bool):
        raise ConfigurationError("logging.level must be a string")
    log_level = raw_level.strip().upper()
    if log_level not in VALID_LOG_LEVELS:
        raise ConfigurationError(
            f"logging.level '{raw_level}' is invalid; must be one of {sorted(VALID_LOG_LEVELS)}"
        )

    log_path = _resolve_relative_path(base_dir, log_dict["path"], "logging.path")

    config = AppConfig(
        input_path=input_path,
        output_path=output_path,
        processing=ProcessingConfig(
            minimum_grade=min_grade,
            skip_invalid=raw_skip,
        ),
        logging=LoggingConfig(
            level=log_level,
            path=log_path,
        ),
    )

    validate_config(config)
    return config
```

Вміст актуального конфігураційного файлу demo_config.yaml, використаного для демонстрації роботи конвеєра: вхідний файл sample_students.csv, вихідний JSON-файл exported_students.json, поріг середнього бала 85.0, активний толерантний режим skip_invalid: true та журналювання на рівні INFO.

```
input:
  path: F:\university\3yr-
1\professional_python\.local_workflow\lab5\sample_students.csv
output:
  path: F:\university\3yr-
1\professional_python\.local_workflow\lab5\exported_students.json
processing:
  minimum_grade: 85.0
  skip_invalid: true
logging:
  level: INFO
  path: F:\university\3yr-1\professional_python\.local_workflow\lab5\application.log
```

7. Налаштування логування та власний контекстний менеджер у reliable_io.py. Функція configure_logging реєструє одночасно FileHandler (із явним UTF-8) та StreamHandler, застосовуючи інформативний формат із міткою часу, рівнем та іменем модуля, а close_logging_handlers коректно закриває активні обробники. Власний контекстний менеджер logged_operation фіксує точний час виконання операції через perf_counter(), логує успіх у блоці else, винятки у блоці except та гарантовано закриває контекст у finally.

```python
@contextmanager
def logged_operation(
    operation: str,
    logger: logging.Logger | None = None,
) -> Iterator[None]:
    log = logger or logging.getLogger("student_manager")
    log.info("%s started", operation)
    start = perf_counter()
    try:
        yield
    except BaseException:
        log.exception("%s failed", operation)
        raise
    else:
        elapsed = perf_counter() - start
        log.info("%s finished in %.6f s", operation, elapsed)
    finally:
        # Гарантоване завершення контексту операції
        log.debug("%s context closed after %.6f s", operation, perf_counter() - start)


def close_logging_handlers(logger: logging.Logger) -> None:
    for h in list(logger.handlers):
        logger.removeHandler(h)
        with suppress(Exception):
            h.close()
```

```python
def configure_logging(
    config: LoggingConfig,
    logger_name: str = "student_manager",
) -> logging.Logger:
    config.path.parent.mkdir(parents=True, exist_ok=True)
    logger = logging.getLogger(logger_name)
    numeric_level = getattr(logging, config.level.upper(), logging.INFO)
    logger.setLevel(numeric_level)

    close_logging_handlers(logger)

    formatter = logging.Formatter(
        "%(asctime)s | %(levelname)s | %(name)s | %(message)s"
    )

    file_handler = logging.FileHandler(config.path, encoding="utf-8")
    file_handler.setLevel(numeric_level)
    file_handler.setFormatter(formatter)
    logger.addHandler(file_handler)

    stream_handler = logging.StreamHandler()
    stream_handler.setLevel(numeric_level)
    stream_handler.setFormatter(formatter)
    logger.addHandler(stream_handler)

    logger.propagate = False
    return logger
```

Фрагмент реального журналу application.log, зафіксованого під час виконання демонстраційного конвеєра. Журнал демонструє фіксацію старту операції, точні попередження WARNING для рядків 4, 5, 6 із зазначенням поля, тривалість виконання та фінальну статистику.

```
2026-10-28 14:18:18,907 | INFO | student_manager | Student processing pipeline
started
2026-10-28 14:18:18,910 | WARNING | student_manager | Invalid record at line 5 (field
'first_name'): First name cannot be empty
2026-10-28 14:18:18,910 | WARNING | student_manager | Invalid record at line 6 (field
'average_grade'): Average grade must be between 0 and 100, got 105.0
2026-10-28 14:18:18,910 | WARNING | student_manager | Invalid record at line 7 (field
'student_id'): Invalid student ID 'abc': must be an integer
2026-10-28 14:18:18,933 | INFO | student_manager | Student processing pipeline
finished in 0.026294 s
```

```
2026-10-28 14:18:18,933 | INFO | student_manager | Total=7, valid=4, invalid=3,
exported=3
```

8. Потоковий читач CSV, політики валідації та збір статистики в reliable_io.py. Функція read_csv_rows є генератором, що читає CSV-файл рядок за рядком (yield) із підтримкою кодування utf-8-sig та суворою перевіркою схеми заголовка EXPECTED_CSV_HEADER. Функція import_students підтримує дві стратегії обробки: у толерантному режимі (skip_invalid: true) невалідні записи логуються, лічильник invalid збільшується, а генератор переходить до наступного рядка; у суворому режимі (skip_invalid: false) перша помилка RecordValidationError негайно перериває імпорт. Функція filter_by_grade здійснює ліниву фільтрацію.

```python
def read_csv_rows(path: Path) -> Iterator[tuple[int, dict[str, str]]]:
    try:
        file = path.open("r", encoding="utf-8-sig", newline="")
    except FileNotFoundError as err:
        raise DataImportError(f"Input file not found: {path}") from err
    except (OSError, PermissionError) as err:
        raise DataImportError(f"Cannot open input file {path}: {err}") from err

    with file:
        reader = csv.reader(file, strict=True)
        try:
            header = next(reader)
        except StopIteration:
            raise DataImportError(f"CSV file is empty: {path}")
        except (UnicodeDecodeError, csv.Error, OSError) as err:
            raise DataImportError(
                f"Failed to read CSV header from {path}: {err}"
            ) from err

        if header != EXPECTED_CSV_HEADER:
            raise DataImportError(
                f"Invalid CSV header schema: expected {EXPECTED_CSV_HEADER}, got {header}"
            )
```

```python
      try:
          for row in reader:
              line_num = reader.line_num
              if len(row) != len(EXPECTED_CSV_HEADER):
                  row_dict = {
                      "_width_error": f"expected {len(EXPECTED_CSV_HEADER)} columns, got {len(row)}"
                  }
              else:
                  row_dict = {
                      EXPECTED_CSV_HEADER[i]: row[i]
                      for i in range(len(EXPECTED_CSV_HEADER))
                  }
              yield (line_num, row_dict)
      except (UnicodeDecodeError, csv.Error, OSError) as err:
          current_line = getattr(reader, "line_num", "unknown")
          raise DataImportError(
              f"Error reading CSV {path} at line {current_line}: {err}"
          ) from err
```

```python
def import_students(
    path: Path,
    *,
    skip_invalid: bool,
    statistics: ImportStatistics,
    logger: logging.Logger | None = None,
) -> Iterator[Student]:
    log = logger or logging.getLogger("student_manager")
    for line_number, row in read_csv_rows(path):
        statistics.total += 1
        try:
            student = parse_student(row, line_number=line_number)
        except RecordValidationError as error:
            statistics.invalid += 1
            log.warning(
                "Invalid record at line %s (field '%s'): %s",
                error.line_number,
                error.field,
                error,
            )
            if skip_invalid:
                continue
            raise
        statistics.valid += 1
        yield student
```

```python
def filter_by_grade(
    students: Iterable[Student],
    minimum_grade: float,
) -> Iterator[Student]:
    for student in students:
        if student.average_grade >= minimum_grade:
            yield student
```

9. Серіалізація в JSON, канонічні ключі та протокол експортера в reliable_io.py. Функція student_to_dict перетворює об'єкт Student у словник із канонічними ключами: student_id, first_name, last_name, group, average_grade. Функція export_students_json записує JSON-масив потоково, не накопичуючи всі словники у пам'яті. Для реалізації розширення підвищеної складності створено інтерфейс StudentExporter на базі typing.Protocol та клас JsonStudentExporter.

```python
def student_to_dict(student: Student) -> dict[str, Any]:
    return {
        "student_id": student.student_id,
        "first_name": student.first_name,
        "last_name": student.last_name,
        "group": student.group,
        "average_grade": student.average_grade,
    }


def export_students_json(students: Iterable[Student], path: Path) -> int:
    count = 0
    with atomic_text_writer(path) as file:
        file.write("[\n")
        first = True
        for student in students:
            if not first:
                file.write(",\n")
            line = json.dumps(student_to_dict(student), ensure_ascii=False, indent=2)
            indented = "\n".join("  " + l for l in line.splitlines())
            file.write(indented)
            first = False
            count += 1
        file.write("\n]\n")
    return count
```

```python
@runtime_checkable
class StudentExporter(Protocol):
    def export(self, students: Iterable[Student], path: Path) -> int: ...


class JsonStudentExporter:
    def export(self, students: Iterable[Student], path: Path) -> int:
        return export_students_json(students, path)
```

Вміст результуючого файлу exported_students.json: експортовано лише студентів із середнім балом >= 85.0 (Олена Коваль - 91.5, Марія Шевченко - 95.2, Андрій Литвин - 89.4; Іван Франко з балом 84.0 відфільтрований). Символи українського алфавіту збережено без екранування (ensure_ascii=False).


```
[
  {
    "student_id": 1,
    "first_name": "Олена",
    "last_name": "Коваль",
    "group": "ФеП-33",
    "average_grade": 91.5
  },
  {
    "student_id": 3,
    "first_name": "Марія",
    "last_name": "Шевченко",
    "group": "ФеП-33",
    "average_grade": 95.2
  },
  {
    "student_id": 7,
    "first_name": "Андрій",
    "last_name": "Литвин",
    "group": "ФеП-33",
    "average_grade": 89.4
  }
]
```


10. Атомарний експорт даних і власний контекстний менеджер atomic_text_writer у reliable_io.py. Для запобігання пошкодженню вихідного файлу при аварійних збоях під час запису запис ведеться у тимчасовий файл NamedTemporaryFile у тому ж каталозі. Перед заміною виконується обов'язковий flush() та скидання буферів ОС на диск через os.fsync(). Заміна цільового файлу виконується неподільною системною операцією os.replace(). У блоці finally при будь-якому обробленому винятку незавершений тимчасовий файл .tmp автоматично видаляється (unlink), а цільовий файл залишається неушкодженим. При цьому не заявляється гарантій повноцінних ACID-транзакцій бази даних чи очищення у разі аварійного вбивства процесу операційною системою (SIGKILL), проте надійно забезпечується неподільна заміна та очищення при штатних і оброблених помилках.

```python
@contextmanager
def atomic_text_writer(path: Path) -> Iterator[IO[str]]:
    resolved_path = path.resolve()
    try:
        resolved_path.parent.mkdir(parents=True, exist_ok=True)
    except OSError as err:
        raise DataExportError(
            f"Cannot create output directory {resolved_path.parent}: {err}"
        ) from err

    temp_path: Path | None = None
    success = False
    committed = False
    try:
        try:
            with tempfile.NamedTemporaryFile(
                mode="w",
                encoding="utf-8",
                newline="",
                dir=resolved_path.parent,
                prefix=f".{resolved_path.name}.",
                suffix=".tmp",
                delete=False,
            ) as temp_file:
                temp_path = Path(temp_file.name)
                yield temp_file
                temp_file.flush()
                os.fsync(temp_file.fileno())
                success = True
        except OSError as err:
            raise DataExportError(
                f"Cannot write temporary file in {resolved_path.parent}: {err}"
            ) from err
```

```python
        if success and temp_path is not None:
            try:
                os.replace(temp_path, resolved_path)
                committed = True
            except OSError as err:
                raise DataExportError(
                    f"Cannot replace destination file {resolved_path}: {err}"
                ) from err
    finally:
        if not committed and temp_path is not None and temp_path.exists():
            try:
                temp_path.unlink(missing_ok=True)
            except OSError:
                logging.getLogger("student_manager").exception(
                    "Cannot remove incomplete temporary output %s", temp_path
                )
```

11. Консольний інтерфейс CLI та коди завершення у lab5_demo.py. Модуль підтримує передачу шляху до конфігураційного файлу через позиційний аргумент командного рядка. У разі успішного виконання функція повертає код 0. При виникненні ApplicationError помилка виводиться у stderr, а програма повертає ненульовий код критичного завершення (exit code 1).

```python
def run_lab5_demo(argv: Sequence[str] | None = None) -> int:
    parser = argparse.ArgumentParser(
        prog="python -m student_manager.lab5_demo",
        description="Run reliable student CSV import and JSON export pipeline",
    )
    parser.add_argument(
        "config",
        nargs="?",
        default="config.yaml",
        help="Path to YAML configuration file (default: config.yaml)",
    )
    args = parser.parse_args(argv)

    try:
        config = load_config(args.config)
        logger = configure_logging(config.logging)
        stats = process_config(config, logger=logger)
        print(
            f"Pipeline completed: total={stats.total}, valid={stats.valid}, "
            f"invalid={stats.invalid}, exported={stats.exported}"
        )
        print(f"Output: {config.output_path}")
        return 0
    except ApplicationError as err:
        sys.stderr.write(f"Application error: {err}\n")
        return 1
    finally:
        # закриваємо хендлери шоб уникнути блокування файлу логів
        import logging

        close_logging_handlers(logging.getLogger("student_manager"))
```

Нижче наведено фактичний вивід реального запуску демонстраційного конвеєра з конфігурацією demo_config.yaml (успішний статус, оброблено 7 рядків, відфільтровано та експортовано 3 записи).

```
PS F:\university\3yr-1\professional_python> python -m student_manager.lab5_demo
.local_workflow/lab5/demo_config.yaml
2026-10-28 14:18:18,907 | INFO | student_manager | Student processing pipeline started
2026-10-28 14:18:18,910 | WARNING | student_manager | Invalid record at line 5 (field 'first_name'): First
name cannot be empty
2026-10-28 14:18:18,910 | WARNING | student_manager | Invalid record at line 6 (field 'average_grade'):
Average grade must be between 0 and 100, got 105.0
2026-10-28 14:18:18,910 | WARNING | student_manager | Invalid record at line 7 (field 'student_id'):
Invalid student ID 'abc': must be an integer
2026-10-28 14:18:18,933 | INFO | student_manager | Student processing pipeline finished in 0.026294 s
2026-10-28 14:18:18,933 | INFO | student_manager | Total=7, valid=4, invalid=3, exported=3
Pipeline completed: total=7, valid=4, invalid=3, exported=3
Output: F:\university\3yr-1\professional_python\.local_workflow\lab5\exported_students.json
```

Перевірка реакції CLI на критичну помилку відсутності конфігураційного файлу: застосунок коректно перехоплює помилку, виводить зрозумілу діагностику в stderr та повертає код завершення 1.

```
PS F:\university\3yr-1\professional_python> python -m student_manager.lab5_demo non_existent_config.yaml
Application error: Configuration file not found: non_existent_config.yaml
```

12. Модульне тестування надійності в tests/test_reliable_io.py, test_config.py та test_reliable_boundaries.py. Для комплексної гарантії стабільності конвеєра розроблено набір із 38 модульних тестів на базі стандартного модуля unittest: 35 тестів охоплюють валідацію полів Student, відсікання значень ID <= 0, перевірку діапазону оцінок [0..100], exception chaining від ValueError, збереження схеми CSV-заголовка, роботу strict/tolerant режимів та валідацію YAML-конфігурації; додаткові 3 тести у test_reliable_boundaries.py покривають

граничні сценарії надійності: логування збою видалення тимчасового файлу без маскування первинної помилки, збереження цільового файлу при збої os.replace та детекцію некоректного CSV як DataImportError.

```python
class TestParseStudent(unittest.TestCase):
    def test_valid_row(self) -> None:
        row = {
            "student_id": "101",
            "first_name": "Іван",
            "last_name": "Франко",
            "group": "ФеП-33",
            "average_grade": "92.5",
        }
        student = parse_student(row, line_number=2)
        self.assertEqual(student.student_id, 101)
        self.assertEqual(student.first_name, "Іван")
        self.assertEqual(student.last_name, "Франко")
        self.assertEqual(student.group, "ФеП-33")
        self.assertEqual(student.average_grade, 92.5)

    def test_invalid_id_not_int(self) -> None:
        row = {
            "student_id": "abc",
            "first_name": "Іван",
            "last_name": "Франко",
            "group": "ФеП-33",
            "average_grade": "92.5",
        }
        with self.assertRaises(RecordValidationError) as ctx:
            parse_student(row, line_number=5)
        self.assertEqual(ctx.exception.line_number, 5)
        self.assertEqual(ctx.exception.field, "student_id")
        self.assertIsInstance(ctx.exception.__cause__, ValueError)
```

```python
    def test_invalid_grade_non_numeric(self) -> None:
        row = {
            "student_id": "1",
            "first_name": "Іван",
            "last_name": "Франко",
            "group": "ФеП-33",
            "average_grade": "wrong",
        }
        with self.assertRaises(RecordValidationError) as ctx:
            parse_student(row, line_number=8)
        self.assertEqual(ctx.exception.field, "average_grade")
        self.assertIsInstance(ctx.exception.__cause__, ValueError)

    def test_invalid_grade_range_and_non_finite(self) -> None:
        for bad_grade in ["-0.5", "100.1", "150", "nan", "inf", "-inf"]:
            row = {
                "student_id": "1",
                "first_name": "Іван",
                "last_name": "Франко",
                "group": "ФеП-33",
                "average_grade": bad_grade,
            }
            with self.assertRaises(RecordValidationError) as ctx:
                parse_student(row, line_number=9)
            self.assertEqual(ctx.exception.field, "average_grade")
```

```python
class ReliableBoundaryTests(unittest.TestCase):
    def test_cleanup_failure_is_logged_without_masking_original_error(self):
        with tempfile.TemporaryDirectory() as folder:
            destination = Path(folder) / "out.json"
            with (
                patch(
                    "student_manager.reliable_io.Path.unlink",
                    side_effect=PermissionError("cleanup denied"),
                ),
                self.assertLogs("student_manager", level="ERROR"),
                self.assertRaisesRegex(ValueError, "original failure"),
                atomic_text_writer(destination),
            ):
                raise ValueError("original failure")

    def test_replace_failure_cleans_temp_and_preserves_destination(self):
        with tempfile.TemporaryDirectory() as folder:
            destination = Path(folder) / "out.json"
            destination.write_text("old", encoding="utf-8")
            with (
                patch(
                    "student_manager.reliable_io.os.replace",
                    side_effect=PermissionError("denied"),
                ),
                self.assertRaises(DataExportError),
                atomic_text_writer(destination) as stream,
            ):
                stream.write("new")
            self.assertEqual(destination.read_text(encoding="utf-8"), "old")
            self.assertEqual(list(Path(folder).glob("*.tmp")), [])
```

```python
    def test_unterminated_csv_quote_is_import_error(self):
        with tempfile.TemporaryDirectory() as folder:
            path = Path(folder) / "input.csv"
            path.write_text(
                'student_id,first_name,last_name,group,average_grade\n1,"Anna,Koval,A,90\n',
                encoding="utf-8",
            )
            with self.assertRaises(DataImportError):
                list(read_csv_rows(path))
```

Результат повного запуску тестового набору: виконано 38 тестів, усі перевірки завершилися зі статусом OK.

```
PS F:\university\3yr-1\professional_python> python -m unittest tests/test_reliable_io.py
tests/test_config.py tests/test_reliable_boundaries.py
........Invalid record at line 3 (field 'average_grade'): Average grade must be between 0 and 100, got
150.0
..Application error: Configuration file not found:
C:\Users\ponti\AppData\Local\Temp\tmpo9dlw3js\missing.yaml
.2026-10-28 14:18:18,185 | INFO | student_manager | Student processing pipeline started
2026-10-28 14:18:18,194 | INFO | student_manager | Student processing pipeline finished in 0.008010 s
2026-10-28 14:18:18,195 | INFO | student_manager | Total=2, valid=2, invalid=0, exported=2
.......2026-10-28 14:18:18,201 | INFO | student_manager | Student processing pipeline started
2026-10-28 14:18:18,209 | INFO | student_manager | Student processing pipeline finished in 0.008706 s
2026-10-28 14:18:18,210 | INFO | student_manager | Total=3, valid=3, invalid=0, exported=2
...................
----------------------------------------------------------------------
Ran 38 tests in 0.351s

OK
Pipeline completed: total=2, valid=2, invalid=0, exported=2
Output: C:\Users\ponti\AppData\Local\Temp\tmpswlh0m6u\out.json
```

13. Перевірка повного циклу серіалізації та десеріалізації (JSON roundtrip). Виконав зворотне читання експортованого файлу exported_students.json: зчитав збережені словники, десеріалізував їх у вихідні екземпляри датакласу Student та порівняв їх зі згенерованими об'єктами. Результат перевірки підтвердив 100% збереження структури полів, типів даних та автентичність українських символів (Олена Коваль, Марія Шевченко, Андрій Литвин, група ФеП-33).

14. Реалізація завдань підвищеної складності (4 розширення для максимальної оцінки). У роботі реалізовано наступні розширені можливості з інструкції:
1) Інтерфейс експортера на базі Protocol: клас StudentExporter оголошено через typing.Protocol із методом export, що забезпечує поліморфізм та слабке зв'язування компонентів конвеєра з конкретним форматом JSON;
2) Вкладена конфігурація (nested configuration): AppConfig інкапсулює окремі датакласи ProcessingConfig та LoggingConfig із підтримкою глибокої структурної валідації типів;
3) Гнучке налаштування логування (configurable logging): функція configure_logging підтримує конфігурацію рівня журналу, одночасний вивід у файл і консоль, створення каталогів та форматування із мітками часу;
4) Контекстний менеджер атомарного запису файлів: atomic_text_writer забезпечує атомарне оновлення цільового файлу через проміжний тимчасовий файл (NamedTemporaryFile), виклик fsync та неподільну системну заміну os.replace(), а також гарантоване видалення тимчасового файлу при оброблених винятках у блоці finally (без надлишкових претензій на повні ACID-гарантії бази даних чи очищення при аварійному примусовому завершенні процесу).

15. Експеримент 1. Дослідження політик обробки помилок: Strict проти Tolerant mode. Для експерименту згенеровано тестовий набір із 1000 валідних та 10 невалідних записів (перший збій у рядку 105). Конвеєр виконано у двох режимах: Strict (skip_invalid: false) та Tolerant (skip_invalid: true). У таблиці нижче наведено порівняння параметрів виконання.

```python
def run_strict_vs_tolerant_experiment(work_dir: Path) -> dict[str, Any]:
    csv_path = work_dir / "strict_vs_tolerant_input.csv"
    generate_synthetic_dataset(
        csv_path,
        valid_count=1000,
        invalid_count=10,
        first_invalid_at=105,
    )

    out_strict = work_dir / "out_strict.json"
    log_strict = work_dir / "strict.log"
    out_tolerant = work_dir / "out_tolerant.json"
    log_tolerant = work_dir / "tolerant.log"

    # 1. суворий режим
    cfg_strict = AppConfig(
        input_path=csv_path,
        output_path=out_strict,
        processing=ProcessingConfig(minimum_grade=0.0, skip_invalid=False),
        logging=LoggingConfig(level="INFO", path=log_strict),
    )
    logger_strict = configure_logging(cfg_strict.logging, logger_name="strict_test")
    strict_error_msg = None
    strict_exit = "success"
    stats_strict = ImportStatistics()
    try:
        process_config(cfg_strict, logger=logger_strict, statistics=stats_strict)
    except ApplicationError as err:
        strict_exit = f"aborted: {err.__class__.__name__}"
        strict_error_msg = str(err)
    finally:
        close_logging_handlers(logger_strict)

    strict_committed = out_strict.exists()
```

```
    # 2. толерантний режим
    cfg_tolerant = AppConfig(
        input_path=csv_path,
        output_path=out_tolerant,
        processing=ProcessingConfig(minimum_grade=0.0, skip_invalid=True),
        logging=LoggingConfig(level="INFO", path=log_tolerant),
    )
    logger_tolerant = configure_logging(
        cfg_tolerant.logging, logger_name="tolerant_test"
    )
    stats_tolerant = ImportStatistics()
    tolerant_exit = "success"
    try:
        process_config(cfg_tolerant, logger=logger_tolerant, statistics=stats_tolerant)
    except ApplicationError as err:
        tolerant_exit = f"aborted: {err}"
    finally:
        close_logging_handlers(logger_tolerant)

    tolerant_committed = out_tolerant.exists()
    tolerant_json_count = 0
    if tolerant_committed:
        with out_tolerant.open("r", encoding="utf-8") as f:
            tolerant_json_count = len(json.load(f))

    return {
        "strict": {
            "processed_total": stats_strict.total,
            "valid": stats_strict.valid,
            "invalid": stats_strict.invalid,
```

```
            "exported": stats_strict.exported,
            "exit_status": strict_exit,
            "error_message": strict_error_msg,
            "file_committed": strict_committed,
        },
        "tolerant": {
            "processed_total": stats_tolerant.total,
            "valid": stats_tolerant.valid,
            "invalid": stats_tolerant.invalid,
            "exported": stats_tolerant.exported,
            "exit_status": tolerant_exit,
            "file_committed": tolerant_committed,
            "records_in_json": tolerant_json_count,
        },
    }
```

Параметр	Strict mode (skip_invalid: false)	Tolerant mode (skip_invalid: true)	Аналіз поведінки системи
Оброблено записів (total)	105 (до першого збою)	1010	Strict негайно перериває обробку на першому збої; Tolerant обробляє повний вхідний набір.
Валідних записів (valid)	104	1000	Усі коректні рядки розпарсено у валідні екземпляри Student без втрати точності.
Невалідних записів (invalid)	1 (перша помилка)	10	Strict підіймає виняток; Tolerant фіксує warning у лог-файл та інкрементує лічильник.
Експортовано у файл (exported)	0	1000	Strict ініціює скасування заміни файлу (0 у JSON); Tolerant зберігає всі 1000 студентів.

Статус виходу застосунку	Аварійний (aborted: RecordValidationError)	Успішний (success, exit code 0)	Strict запобігає появі пошкоджених файлів; Tolerant максимізує обсяг завантажених даних.
Стан цільового JSON-файлу	Файл не створено (скасовано заміну)	Створено валідний масив (1000 об'єктів)	Контекстний менеджер atomic_text_writer гарантує відсутність частково записаного файлу.

Аналіз результатів експерименту 1:

- Strict mode зупиняється на першому ж невалідному рядку (оброблено лише 105 записів). Завдяки неподільній заміні файлу жоден рядок не потрапляє у вихідний файл (0 експортовано), що захищає цільовий файл від часткового запису за принципом 'все-або-нічого'. Цей режим є обов'язковим для банківських проводок, фінансових розрахунків та міграцій критичних баз даних;
- Tolerant mode успішно опрацьовує всі 1010 рядків файлу, фіксує 10 попереджень у логах та експортує 1000 валідних записів. Цей режим є оптимальним для аналітичних конвеєрів та збору метрик, де поодинокі дефекти не повинні блокувати основний потік даних.

16. Експеримент 2. Порівняння швидкодії та використання пам'яті: Streaming проти Eager. Дослідження проведено для синтетичних наборів даних розміром 10 000, 100 000 та 500 000 записів. Eager-варіант завантажує весь файл у список через list(reader), матеріалізує об'єкти та списокову фільтрацію. Streaming-варіант обробляє дані через генераторний конвеєр import_students та filter_by_grade. Пікове споживання пам'яті вимірювалося стандартним модулем tracemalloc.

```python
def run_streaming_vs_eager_experiment(
    work_dir: Path,
    sizes: tuple[int, ...] = (10_000, 100_000, 500_000),
) -> list[dict[str, Any]]:
    results = []

    for size in sizes:
        csv_path = work_dir / f"bench_{size}.csv"
        # генерація за межами вимірювань
        with csv_path.open("w", encoding="utf-8", newline="") as f:
            f.write("student_id,first_name,last_name,group,average_grade\n")
            batch = []
            for i in range(1, size + 1):
                batch.append(
                    f"{i},Студент{i % 100},Прізвище{i % 100},ФеП-33,{(60 + i % 40):.1f}\n"
                )
                if len(batch) >= 10_000:
                    f.writelines(batch)
                    batch.clear()
            if batch:
                f.writelines(batch)
                batch.clear()
```

```python
        # 1. Eager вимірювання
        gc.collect()
        tracemalloc.start()
        t0 = perf_counter()

        # eager завантажує весь список у пам'ять
        raw_rows = []
        for line_num, r in read_csv_rows(csv_path):
            raw_rows.append((line_num, r))
        eager_students = [parse_student(r, line_number=ln) for ln, r in raw_rows]
        eager_filtered = [s for s in eager_students if s.average_grade >= 75.0]
        eager_count = len(eager_filtered)

        t_eager = perf_counter() - t0
        _, peak_eager = tracemalloc.get_traced_memory()
        tracemalloc.stop()
```

```python
        # 2. Streaming вимірювання
        del raw_rows, eager_students, eager_filtered
        gc.collect()
        tracemalloc.start()
        t0 = perf_counter()

        # streaming обробляє елементи по одному через генератор
        stats = ImportStatistics()
        stream_gen = import_students(csv_path, skip_invalid=True, statistics=stats)
        filtered_gen = filter_by_grade(stream_gen, 75.0)
        stream_count = sum(1 for _ in filtered_gen)

        t_stream = perf_counter() - t0
        _, peak_stream = tracemalloc.get_traced_memory()
        tracemalloc.stop()

        # видаляємо великий тестовий csv шоб не займати дисковий простір
        csv_path.unlink(missing_ok=True)

        results.append(
            {
                "dataset_records": size,
                "eager_filtered_count": eager_count,
                "stream_filtered_count": stream_count,
                "eager_time_sec": round(t_eager, 4),
                "streaming_time_sec": round(t_stream, 4),
                "eager_peak_ram_bytes": peak_eager,
                "streaming_peak_ram_bytes": peak_stream,
                "eager_peak_ram_mb": round(peak_eager / (1024 * 1024), 2),
                "streaming_peak_ram_mb": round(peak_stream / (1024 * 1024), 2),
                "ram_saving_factor": round(peak_eager / max(1, peak_stream), 2),
            }
        )

    return results
```

Обсяг (rows)	Eager час (с)	Streaming час (с)	Eager пікова RAM (МіБ)	Streaming пікова RAM (МіБ)	Економія RAM
10 000	0.85 с	0.79 с	7.23 МіБ	0.07 МіБ	105.82x
100 000	9.69 с	8.55 с	72.48 МіБ	0.07 МіБ	1067.42x
500 000	46.67 с	41.98 с	362.98 МіБ	0.07 МіБ	5348.14x

Аналіз результатів експерименту 2:

- Пікова пам'ять в Eager-варіанті масштабується лінійно O(n): від 7.23 МіБ для 10k до 72.48 МіБ для 100k та 362.98 МіБ для 500k рядків;
- Пікова пам'ять у Streaming-варіанті залишається строго константною O(1) і становить лише ~0.07 МіБ (~69.50 КіБ) незалежно від обсягу набору;
- Фактор економії оперативної пам'яті зростає пропорційно обсягу: з 105.82x для 10 000 записів (1067.42x для 100 000) до 5348.14x для 500 000 записів;

- Час виконання потокового генератора (0.79 с, 8.55 с, 41.98 с) є дещо кращим за eager-варіант (0.85 с, 9.69 с, 46.67 с), оскільки інтерпретатор уникає виділення гігантських проміжних масивів та накладних витрат збирача сміття (GC);
- Слід підкреслити, що модуль tracemalloc вимірює виділення пам'яті під об'єкти середовищем Python (heap allocations), а не загальний обсяг RAM або Resident Set Size (RSS) процесу на рівні операційної системи.

17. Експеримент 4. Дослідження атомарного виводу при штучному перериванні експорту. У ході експерименту створено цільовий файл із початковим захищеним вмістом. Під час потокового запису у генераторі штучно викликано виняток RecordValidationError на другому рядку. Результат перевірки підтвердив:
1) Попередній вміст цільового файлу залишився повністю неушкодженим;
2) Тимчасовий файл .tmp було негайно видалено в блоці finally (список залишкових файлів порожній: []);
3) Помилку зареєстровано, а застосунок завершив роботу без залишення пошкодженого виводу.

```python
def run_atomic_rollback_experiment(work_dir: Path) -> dict[str, Any]:
    out_file = work_dir / "atomic_target.json"
    original_content = '{"protected": "original data should not be corrupted"}'
    out_file.write_text(original_content, encoding="utf-8")

    def failing_stream() -> Iterator[Student]:
        yield Student(
            first_name="Олена",
            last_name="Коваль",
            group="ФеП-33",
            average_grade=95.0,
            student_id=1,
        )
        # штучна помилка всередині генератора
        raise RecordValidationError(
            "Fatal lazy stream error", line_number=2, field="generator"
        )

    error_name = ""
    try:
        export_students_json(failing_stream(), out_file)
    except ApplicationError as err:
        error_name = err.__class__.__name__

    after_content = out_file.read_text(encoding="utf-8")
    leftover_tmps = [p.name for p in out_file.parent.glob("*.tmp")]

    return {
        "target_file": str(out_file),
        "exception_raised": error_name,
        "content_preserved": after_content == original_content,
        "leftover_temporary_files": leftover_tmps,
    }
```

18. Експеримент 5. Перевірка обробки некоректної конфігурації за 6 сценаріями. Досліджено реакцію завантажувача конфігурації на: відсутній файл конфігурації, синтаксичну помилку YAML, відсутність обов'язкової секції input, неприпустимий бал 150.0, відсутній вхідний CSV-файл та неіснуючий вихідний каталог. Результати перевірки зведено в таблицю нижче.

```
def run_config_errors_experiment(work_dir: Path) -> list[dict[str, Any]]:
    valid_csv = work_dir / "valid_students.csv"
    valid_csv.write_text(
        "student_id,first_name,last_name,group,average_grade\n1,A,B,C,90\n",
        encoding="utf-8",
    )

    cases = [
        ("missing_config", work_dir / "absent_config.yaml", None),
        ("malformed_yaml", work_dir / "malformed.yaml", "input: [unclosed"),
        (
            "missing_input_section",
            work_dir / "no_input.yaml",
            "output:\n  path: out.json\nprocessing:\n  minimum_grade: 60\n  skip_invalid: true\nlogging:\n  level: INFO\n  path: app.log\n",
        ),
        (
            "grade_out_of_range_150",
            work_dir / "grade_150.yaml",
            f"input:\n  path: {valid_csv.name}\noutput:\n  path: out.json\nprocessing:\n  minimum_grade: 150.0\n  skip_invalid: true\nlogging:\n  level: INFO\n  path: app.log\n",
        ),
        (
            "missing_input_file",
            work_dir / "absent_data.yaml",
            "input:\n  path: not_found.csv\noutput:\n  path: out.json\nprocessing:\n  minimum_grade: 60.0\n  skip_invalid: true\nlogging:\n  level: INFO\n  path: app.log\n",
        ),
        (
            "missing_output_dir",
            work_dir / "create_out_dir.yaml",
            f"input:\n  path: {valid_csv.name}\noutput:\n  path: deep/nested/dir/out.json\nprocessing:\n  minimum_grade: 60.0\n  skip_invalid: true\nlogging:\n  level: INFO\n  path: app.log\n",
        ),
    ]
```

```
    results = []
    for label, cfg_path, content in cases:
        if content is not None:
            cfg_path.write_text(content, encoding="utf-8")

        status = ""
        error_type = ""
        try:
            cfg = load_config(cfg_path)
            # перевіряємо чи для missing_output_dir процес створення каталогу проходить успішно
            if label == "missing_output_dir":
                logger = configure_logging(cfg.logging, logger_name="cfg_test")
                try:
                    process_config(cfg, logger=logger)
                    status = (
                        f"success_directory_created_{cfg.output_path.parent.exists()}"
                    )
                finally:
                    close_logging_handlers(logger)
            else:
                status = "loaded_unexpectedly"
        except ConfigurationError as err:
            status = "rejected_as_expected"
            error_type = f"{err.__class__.__name__}: {err}"

        results.append(
            {
                "case": label,
                "config_file": cfg_path.name,
                "status": status,
                "error_diagnostic": error_type,
            }
        )
    return results
```

Тестовий сценарій	Конфігураційний файл	Діагностичне повідомлення	Статус	Поведінка застосунку
missing_config	absent_config.yaml	Configuration file not found: C:\Users\ponti\AppData\Local\Temp\tmp7vcjn...	rejected_as_expected	Блокування конвеєра: файл відсутній
malformed_yaml	malformed.yaml	Invalid YAML configuration syntax in C:\Users\ponti\AppData\Local\Temp\t...	rejected_as_expected	Перехоплення YAMLError з позицією
missing_input_section	no_input.yaml	Configuration is missing required section 'input' or it is not a mapping	rejected_as_expected	Структурна валідація секцій словника
grade_out_of_range_150	grade_150.yaml	processing.minimum_grade must be a finite number between 0 and 100	rejected_as_expected	Діапазонний контроль [0..100]
missing_input_file	absent_data.yaml	Input file does not exist: C:\Users\ponti\AppData\Local\Temp\tmp7vcjn8iz...	rejected_as_expected	Перевірка path.exists() до читання
missing_output_dir	create_out_dir.yaml	Помилки немає, каталог створено успішно	success_directory_created_True	mkdir(parents=True) створює папки

Аналіз результатів експерименту 5 підтверджує надійну роботу принципу fail-fast: усі критичні помилки налаштувань перехоплюються на етапі ініціалізації до виділення ресурсів чи відкриття файлів даних. Водночас відсутність вихідного каталогу не є фатальною помилкою - застосунок автоматично створює вкладені каталоги через mkdir().

19. Теоретичні засади надійної розробки: Path, with, try/except/else/finally та парадигма EAFP. Сучасний професійний Python-код базується на таких фундаментальних принципах:
- Об'єкти pathlib.Path абстрагують роботу з файловою системою, захищають від помилок роздільників Windows/POSIX та усувають проблеми відносних шляхів через метод resolve();
- Конструкція with гарантує детерміноване звільнення дескрипторів файлів, сокетів та блокувань за стандартом RAII;
- Повна форма try/except/else/finally забезпечує прозорий життєвий цикл операції: перехоплення лише очікуваних винятків, виконання блоку else лише при безпомилковому результаті та гарантоване очищення ресурсів у finally;
- Парадигма EAFP дозволяє писати лаконічний і швидкий код, усуваючи стан гонитви (race conditions) під час перевірки файлів, а принцип fail-fast захищає програму від виконання дорогих операцій з некоректними початковими параметрами.

Контрольні запитання

1. Що таке exception?

Exception - об'єкт у Python, який сигналізує про виникнення нештатної ситуації або помилки під час виконання програми і перериває звичайний потік інструкцій.

2. Для чого використовується try?

Блок try використовується для виокремлення ділянки коду, під час виконання якої можуть виникнути потенційні винятки, за якими встановлюється нагляд.

3. Для чого except?

Блок except перехоплює виняток певного типу, що виник у блоці try, і виконує код його обробки або відновлення роботи програми.

4. Коли виконується else після try?

Блок else виконується лише тоді, коли блок try завершився успішно і в ньому не виникло жодного винятку.

5. Коли виконується finally?

Блок finally виконується обов'язково і завжди після блоків try та except, незалежно від того, чи стався виняток, чи код завершився успішно.

6. Для чого використовується raise?

Оператор raise використовується для явного підняття об'єкта винятку та передачі сигналу про помилку вище по стеку викликів.

7. Що означає raise без аргументів?

raise без аргументів використовується всередині блоку except і повторно підіймає поточний активний виняток без зміни його типу та стеку виклику.

8. Чому не рекомендується except: pass?

Конструкція except: pass приховує будь-які неочікувані помилки (зокрема помилки синтаксису чи друкарські помилки в іменах змінних), унеможливлюючи діагностику та зневадження програми.

9. Чому потрібно перехоплювати конкретні exception types?

Перехоплення конкретних типів дозволяє обробляти лише очікувані нештатні ситуації, не маскуючи фатальні системні збої та системні винятки на кшталт KeyboardInterrupt чи MemoryError.

10. Що таке exception hierarchy?

Це деревоподібна структура успадкування класів винятків від базового BaseException (і зазвичай Exception), що дозволяє групувати споріднені помилки за категоріями.

11. Для чого створювати custom exceptions?

Власні винятки дозволяють визначити специфічні для предметної області типи помилок, додавати контекстні атрибути (номер рядка, поле) та семантично розділяти бізнес-помилки від системних.

12. Що таке exception chaining?

Це механізм збереження первинного винятку як причини нового винятку, що дозволяє відстежувати повний ланцюжок виникнення проблеми під час її перетворення.

13. Що означає raise ... from ...?

Конструкція raise NewException from original_error явно встановлює original_error як атрибут __cause__ для NewException, фіксуючи прямий причинно-наслідковий зв'язок у traceback.

14. Яку перевагу дає chaining для debugging?

Chaining зберігає повний первинний контекст помилки (наприклад, конкретне значення, що не конвертувалося у float), запобігаючи втраті деталей при переході на рівень доменних винятків.

15. Що означає EAFP?

EAFP (Easier to Ask for Forgiveness than Permission) - ідіома Python, що полягає у спробі виконати операцію без попередніх перевірок та перехопленні винятку у разі виникнення помилки.

16. Що означає LBYL?

LBYL (Look Before You Leap) - підхід, за якого перед виконанням операції здійснюються явні попередні перевірки умов (наприклад, перевірка if key in dict перед доступом до елемента).

17. Що таке fail-fast?

Fail-fast - принцип проєктування, за якого система негайно перериває виконання при виявленні першої некоректної конфігурації чи критичної помилки, не починаючи ресурсомістку роботу.

18. Де доцільно застосовувати fail-fast?

Fail-fast доцільно застосовувати під час завантаження та валідації конфігурації, перевірки доступності критичних ресурсів і цілісності вхідних параметрів на старті застосунку.

19. Що таке context manager?

Context manager - об'єкт, який керує життєвим циклом ресурсу, гарантуючи виконання дій з ініціалізації при вході та обов'язкове звільнення ресурсів при виході з блоку коду.

20. Для чого використовується with?

Оператор with автоматизує виклик методів context manager, забезпечуючи детерміноване закриття файлів чи звільнення блокувань навіть у разі виникнення винятків.

21. Які methods утворюють context manager protocol?

Протокол context manager утворюють два спеціальні магічні методи: __enter__() для ініціалізації ресурсу та __exit__() для його фіналізації та обробки винятків.

22. Що робить __enter__()?

Метод __enter__() налаштовує контекст виконання, захоплює необхідний ресурс і повертає об'єкт, який присвоюється змінній після ключового слова as.

23. Що робить __exit__()?

Метод __exit__() звільняє ресурс при виході з блоку with, отримує інформацію про виняток (якщо він стався) та повертає True, якщо виняток слід придушити, або False для його поширення.

24. Чи закриває with файл при exception?

Так, оператор with гарантовано закриває відкритий файл, оскільки метод __exit__() викликається середовищем виконання Python завжди, навіть якщо у блоці with виник виняток.

25. Для чого contextlib.contextmanager?

Декоратор contextlib.contextmanager дозволяє створювати context manager на основі функції-генератора без необхідності вручну писати окремий клас із методами __enter__ та __exit__.

26. Яку роль має yield у generator-based context manager?

Інструкція yield розділяє контекст: код до yield виконується як метод __enter__(), значення після yield передається у блок with, а код після yield (зазвичай у finally) виконується як __exit__().

27. Що таке pathlib.Path?

pathlib.Path - об'єктно-орієнтоване представлення файлового шляху у стандартній бібліотеці Python, що інкапсулює кросплатформні операції над файлами та каталогами.

28. Чому pathlib кращий за ручну конкатенацію шляхів?

pathlib автоматично враховує роздільники конкретної операційної системи через оператор /, нормалізує шляхи, запобігає помилкам конкатенації рядків і надає багатий набір готових методів перевірки.

29. Для чого задається encoding="utf-8"?

Явна передача encoding="utf-8" гарантує коректне читання та запис символів національних алфавітів і усуває залежність програми від системної кодової сторінки операційної системи (наприклад, Windows-1251).

30. Що таке CSV?

CSV (Comma-Separated Values) - текстовий формат для зберігання табличних даних, де кожен рядок відповідає запису таблиці, а окремі поля відокремлюються комами чи іншими роздільниками.

31. Для чого csv.reader?

csv.reader використовується для послідовного потокового читання рядків CSV-файлу у вигляді списків текстових значень з автоматичним урахуванням лапок та екранування.

32. Для чого csv.DictReader?

csv.DictReader зчитує рядки CSV-файлу і автоматично перетворює кожен рядок у словник dict, використовуючи перший рядок файлу як назви ключів.

33. Чим DictReader зручний для structured data?

DictReader дозволяє звертатися до полів за зрозумілими іменами колонок замість числових індексів, що підвищує читабельність коду та стійкість до зміни порядку стовпців.

34. Для чого csv.DictWriter?

csv.DictWriter використовується для запису структурованих даних у формат CSV зі словників, автоматично впорядковуючи поля згідно із заздалегідь визначеним списком fieldnames.

35. Що таке JSON?

JSON (JavaScript Object Notation) - текстовий формат обміну структурованими даними, побудований на парах ключ-значення та впорядкованих списках.

36. Що таке serialization?

Серіалізація - процес перетворення внутрішніх об'єктів або структур даних мови програмування у послідовність байтів чи текстовий формат, придатний для збереження у файлі або передачі мережею.

37. Що таке deserialization?

Десеріалізація - зворотний процес реконструкції вихідного програмного об'єкта чи структури даних із серіалізованого текстового чи бінарного представлення.

38. Чим json.dump() відрізняється від json.dumps()?

json.dump() записує серіалізовані дані безпосередньо у відкритий файловий потік, тоді як json.dumps() повертає результат серіалізації у вигляді звичайного рядка str у пам'яті.

39. Чим json.load() відрізняється від json.loads()?

json.load() зчитує та десеріалізує JSON безпосередньо з файлового об'єкта, а json.loads() приймає рядок або байти із пам'яті та перетворює їх на структуру Python.

40. Що таке JSONDecodeError?

JSONDecodeError - виняток, що виникає при спробі десеріалізації синтаксично некоректного текстового документа JSON (наприклад, незакриті дужки чи зайві коми).

41. Для чого використовується YAML?

YAML використовується як людинозручний формат опису конфігураційних файлів, що підтримує коментарі, ієрархічну вкладеність через відступи та різноманітні скалярні типи.

42. Чому рекомендується yaml.safe_load()?

yaml.safe_load() обмежує парсер лише стандартними безпечними типами YAML, захищаючи застосунок від виконання довільного коду, можливого через небезпечний повний yaml.load().

43. Які типові сценарії використання JSON?

JSON є стандартом для REST API, обміну даними між мікросервісами, збереження стану кешів та експорту структурованих даних для машинної обробки.

44. Які типові сценарії використання YAML?

YAML найчастіше використовується для написання конфігураційних файлів застосунків, налаштувань інфраструктури, CI/CD конвеєрів та маніфестів Kubernetes.

45. Що таке configuration file?

Це зовнішній файл (наприклад, у форматі YAML чи JSON), що містить параметри роботи програми (шляхи до файлів, пороги, рівні логування), відокремлені від вихідного коду.

46. Чому configuration краще не hard-code у Python?

Відокремлення конфігурації від коду дозволяє змінювати поведінку програми, середовище виконання чи порогові параметри без повторного редагування, компіляції чи розгортання коду.

47. Що потрібно перевіряти у configuration?

У конфігурації необхідно перевіряти наявність усіх обов'язкових секцій, коректність типів значень, допустимі межі чисел (діапазони) та існування вказаних файлів і каталогів.

48. Що таке logging?

Logging - стандартизований процес програмного фіксування подій, інформаційних повідомлень, попереджень та помилок, що виникають під час виконання застосунку, із зазначенням часу та контексту.

49. Чим logging відрізняється від print()?

Модуль logging підтримує рівні серйозності подій, автоматичне форматування із часовими мітками, фільтрацію та одночасне спрямування записів у файл і консоль без зміни коду.

50. Які існують logging levels?

Основними стандартними рівнями логування за зростанням пріоритету є: DEBUG, INFO, WARNING, ERROR та CRITICAL.

51. Для чого logger.info()?

logger.info() використовується для фіксації штатних очікуваних подій процесу (наприклад, старт конвеєра, завершення кроку імпорту, підсумкова кількість оброблених записів).

52. Для чого logger.warning()?

logger.warning() сигналізує про виникнення нештатних, але не фатальних ситуацій (наприклад, пропуск конкретного некоректного рядка CSV у tolerant режимі).

53. Для чого logger.error()?

logger.error() фіксує серйозні помилки, через які конкретна операція чи запис не змогли виконатися, але застосунок загалом може продовжувати роботу.

54. Для чого logger.exception()?

logger.exception() записує повідомлення про помилку на рівні ERROR і автоматично додає до запису повний стек викликів (traceback) поточного активного винятку.

55. Що містить traceback?

Traceback містить послідовність викликів функцій від точки старту до місця виникнення винятку, включаючи шляхи до файлів, номери рядків та текстовий опис кожної інструкції.

56. Чому secrets не можна записувати в logs?

Запис паролів, ключів доступу та персональних даних у логи створює критичну вразливість витоку конфіденційної інформації через незахищені журнали чи сторонні системи моніторингу.

57. Що таке streaming I/O?

Streaming I/O - потокова обробка даних порціями чи окремими рядками за допомогою генераторів без завантаження всього масиву інформації в оперативну пам'ять одночасно.

58. Яка перевага streaming для великих files?

Потокова обробка зберігає константне споживання оперативної пам'яті O(1), дозволяючи обробляти файли будь-якого розміру, що перевищують доступний обсяг RAM.

59. Що таке eager file loading?

Eager loading - підхід, за якого весь вміст файлу або всі його записи негайно зчитуються та матеріалізуються в пам'яті у вигляді повного списку перед початком обробки.

60. Чому list(reader) може бути небажаним?

Виклик list(reader) на великих наборах даних спричиняє раптовий сплеск використання пам'яті (Memory Spike) і може призвести до аварійного завершення програми через MemoryError.

61. Що таке strict validation mode?

Strict mode - суворий режим валідації, за якого перша виявлена помилка у даних негайно перериває обробку та скасовує створення вихідного файлу.

62. Що таке tolerant mode?

Tolerant mode - толерантний режим, за якого некоректні рядки фіксуються у логах і підраховуються лічильником помилок, а обробка решти валідних даних продовжується.

63. Коли доцільно використовувати strict mode?

Strict mode необхідний у фінансових системах, транзакційному обліку та під час міграції баз даних, де будь-яка втрата чи неточність даних є неприпустимою.

64. Коли tolerant mode?

Tolerant mode доцільний під час збору аналітики, обробки журналів подій чи телеметрії, де важливіше отримати максимум доступних даних, пожертвувавши поодинокими збійними записами.

65. Що таке atomic write?

Atomic write - підхід до запису файлу через проміжний тимчасовий файл із фінальною заміною цільового файлу однією неподільною системною операцією.

66. Для чого використовується temporary file?

Тимчасовий файл акумулює вихідні дані під час роботи генератора; якщо процес перерветься аварійно, цільовий файл залишиться неушкодженим.

67. Що може статися при звичайному output write та аварійному завершенні?

При звичайному відкритті файлу у режимі "w" його попередній вміст одразу очищується, а при аварійному збої на диску залишається пошкоджений або частково записаний файл.

68. Навіщо видаляти temporary file після error?

Видалення тимчасового файлу у разі виникнення помилки запобігає засміченню дискового простору незавершеними сміттєвими файлами (.tmp).

69. Що таке JSON Lines?

JSON Lines (JSONL) - текстовий формат зберігання структурованих даних, у якому кожен окремий рядок файлу є повністю самостійним валідним об'єктом JSON.

70. Чому JSON Lines зручний для streaming?

JSON Lines дозволяє читати, валідувати та записувати кожен об'єкт окремо рядковим потоком без необхідності тримати відкриті дужки масиву чи парсити весь документ цілком.

Висновок:

У ході виконання лабораторної роботи №5 створив надійний модуль імпорту, експорту та конфігурації даних для системи обліку студентів (варіант 1). Реалізував власну доменну ієрархію винятків (ApplicationError, ConfigurationError, DataError, RecordValidationError, DataExportError) із підтримкою збереження первинного контексту через exception chaining (from err). Застосував принцип fail-fast для завантаження та суворої валідації конфігурації YAML, що унеможливлює некоректний запуск конвеєра. Реалізував потокове читання CSV через генератор, підтримку strict та tolerant режимів валідації з фіксацією помилок у журналі logging та збором статистики. Створив власний контекстний менеджер atomic_text_writer на базі NamedTemporaryFile, fsync та os.replace, що гарантує неподільність оновлення цільового файлу та очищення тимчасового файлу при оброблених винятках у блоці finally без залишення пошкодженого виводу при збоях у потоковому генераторі (очищення потребує виконання finally). У серії з 5 практичних експериментів підтвердив теоретичні положення: на вибірці з 1010 записів tolerant-режим успішно експортував 1000 валідних об'єктів у JSON, зафіксувавши 10 помилок, тоді як strict-режим здійснив атомарне скасування заміни файлу при першому ж збої; бенчмарк на 10 000, 100 000 та 500 000 записів продемонстрував константне споживання пам'яті генератором (0.07 МіБ проти 362.98 МіБ у списку), досягнувши колосальної економії пам'яті у 5348.14 раза (виміряно через tracemalloc для heap allocations Python); експеримент зі штучним збоєм підтвердив повне збереження цільового файлу без залишкових .tmp-файлів. Усі компоненти архітектури верифіковано повним набором із 38 модульних тестів (35 тестів логіки й конфігурації + 3 граничні тести очищення й парсингу).